onlyness-axi: Portability Review of Tasks 6-8

Oct 20, 2026 · @John Stansbury

Verdict

Tasks 6 to 8 will work for any agent that can run a shell command and read its output, once one bug is fixed. The plan keeps state, gates and the 3-turn coach budget in the CLI, so a weaker model cannot skip Act 0 or loop forever. Reviewed against the pushed plan and design spec in `coach-punk/onlyness-axi`, which contain no code yet.

The blocker: `poll` cannot see the server's events

The event queue in `events.ts` is in-memory module state, and `waitForEvent` runs in-process. But `start` and `poll` are separate CLI invocations, so they are separate Node processes. The server inside the `start` process receives the human's submit and pushes it into its own memory. The later `poll` process starts with an empty queue and times out.

The Task 6 test passes anyway because it runs the server and `waitForEvent` in one process. The Task 8 smoke test only checks `start` and `list`, so nothing in the plan would catch this.

Fix: make events durable.

- The server appends each event to `.onlyness/<id>/events.jsonl` with an incrementing `seq`.
- `poll --since <seq>` tails that file and never talks to the server.
- If the server is dead, `poll` relaunches it with a `serve <id>` command, since all state is on disk.

This also fixes events lost when the human submits while the agent is still thinking, and servers killed between turns by sandboxes that stop background processes.

Portability gaps by harness type

Seven gaps beyond the blocker, ordered by how many harnesses they affect.

Gap	Who it breaks	Fix
Human's browser unreachable	Cloud agents (Copilot coding agent, Codespaces, Claude Code on the web)	Add `--no-open`, print the URL prominently, note port forwarding
No chat-only path	Any agent without a browser surface	Add `submit <session> --step … --json …` so the agent relays answers typed in chat
60-second blocking `poll`	Harnesses that cut commands at 30s or buffer output	Default to about 25s, make it `--wait N`, keep the `event: timeout` retry hint
`session.json` written by server and CLI	Everyone, as a race	Atomic writes (temp file, then rename) plus a `version` field, or one writer
Localhost accepts any POST	Any browser the human uses	Bind to loopback only; random per-session token in the URL; check `Origin`
Page re-rendered per step	Humans mid-typing	Static shell rendered from `/api/session` JSON, with drafts kept client-side
Human free text returned in `poll` output	Any model, as a prompt-injection path	Fence it in a labeled `payload` block, truncate, `--full` for the rest

What already helps weaker models

The CLI owns state, advance gates and the coach budget, so a smaller model cannot skip Act 0 or truthmode, and cannot coach forever. One addition would help further: `poll` output should say exactly what to do next, for example `coachJob: clarify` plus one line on what that job means. Today it reports only how many turns were used, which leaves a smaller model guessing.

Skill discovery beyond Claude

Skills load automatically only in Claude. Add an `onlyness-axi init` command that writes the same short instructions as an `AGENTS.md` block, which Copilot, Cursor and other agents that read that file will pick up. This is cheap because the plan already keeps the skill thin and serves guidance from `--help` and `playbook`.

A test that proves it

A "dumb driver" script that imports none of the project's code and follows only the `next[]` lines in CLI output. If it passes, the loop works for any agent.

1. Run `onlyness-axi start` with `ONLYNESS_NO_BROWSER=1` and read the session id and URL from stdout.
2. POST a submit to the printed URL over HTTP.
3. Run `onlyness-axi poll <id> --since 0` as a separate process and check it returns the submit event.
4. Repeat the submit while no `poll` is running, then confirm the next `poll --since <seq>` still returns it.
5. Kill the server process and confirm `poll` relaunches it.

Recommended next step

Rewrite tasks 6 to 8 in the plan file with the durable event log, a `submit` command, a `serve` command, token auth and a static surface, and add the cross-process test above. Keep it as a draft in `docs/superpowers/plans/2026-10-19-onlyness-axi.md` for review before any code is written.